

Alocação de Computação de Inferência na Síntese de Programas com Modelos de Linguagem

Anonymous submission

Resumo

Um orçamento fixo de chamadas a um modelo de linguagem pode ser gasto na geração de N candidatos independentes ou na revisão de um único candidato com feedback. A literatura relata ganhos e também falhas para a segunda via, e separar os dois casos é difícil porque, sem informação externa, o revisor não sabe que errou. Concedemos a essa via uma vantagem indisponível em produção. Um agente Crítico recebe o gabarito do par de teste e aponta contradições ao agente Gerador, sem escrever código e sem participar da escolha da resposta final. Em 270 tarefas do ARC-AGI-1 sob orçamento igualado, **nenhuma** das 5 comparações pré-registradas é significativa. Medimos também que cerca de metade das vitórias de qualquer condição vem do primeiro programa gerado, antes de qualquer crítica ou reamostragem, o que reduz a fração da amostra em que a comparação de fato discrimina e inverte a ordenação entre as condições quando descontado. A ordenação observada nas 270 tarefas se reproduz em 8% das subamostras de 30.

1 Introdução

Ganhos recentes em tarefas de raciocínio dependem cada vez mais da computação gasta no momento da inferência (Snell et al. 2024; Brown et al. 2024). Um orçamento fixo de chamadas ao modelo admite formas distintas de gasto, com consequência direta de custo.

Uma alternativa é diversificar, gerando N programas independentes e selecionando o melhor por um verificador barato. A prática aparece na avaliação por *pass@k*, em que se amostram k soluções e basta uma correta (Chen et al. 2021), e no AlphaCode (Li et al. 2022). A outra é iterar sobre um único candidato, gastando o mesmo orçamento em revisões guiadas por feedback, como fazem *self-debugging* (Chen et al. 2024), SELF-REFINE (Madaan et al. 2023) e REFLEXION (Shinn et al. 2023).

A evidência sobre a via iterativa é contraditória, com ganhos expressivos nos trabalhos originais e perdas em réplicas controladas (Huang et al. 2024; Stechly, Marquez, and Kambhampati 2023; Olausson et al. 2024). Uma explicação atribui a discrepância à falta de informação externa sobre o erro, que

levaria o modelo a reafirmar a resposta já produzida. Sob essa hipótese, o fator limitante está no canal de feedback.

Testamos essa hipótese dando ao canal de feedback a melhor informação disponível. O solucionador tem dois agentes. O **Gerador** recebe os pares de demonstração e propõe uma regra e um programa. O **Crítico** examina esse candidato e devolve contradições em prosa, sem escrever código. Nas condições que o concedem, o Crítico recebe também o gabarito do par de teste, que o Gerador nunca vê, e chamamos esse acesso de **oráculo**. Nenhum sistema autônomo teria essa informação, de modo que a comparação favorece a via iterativa por construção. Se ela não vencer nem assim, a explicação pelo canal de feedback perde força.

Avaliamos no ARC-AGI-1 (Chollet 2019), em que cada tarefa traz pares entrada → saída de demonstração e exige inferir a transformação subjacente. A resposta admite verificação exata, e os pares de demonstração funcionam como verificador barato que todas as condições usam sem acesso ao gabarito.

Nenhuma das cinco comparações pré-registradas separa as condições. A decomposição das vitórias explica parte da dificuldade de medir essa diferença. Cerca de metade dos acertos de qualquer condição vem do primeiro programa gerado, antes de qualquer crítica ou reamostragem, o que reduz a parte da amostra em que as condições podem de fato ser distinguidas.

2 Trabalho Relacionado

A via iterativa realimenta o modelo com o resultado da execução (Chen et al. 2024) ou com crítica que ele mesmo gera (Madaan et al. 2023; Shinn et al. 2023). Huang et al. (2024) argumentam que modelos não corrigem raciocínio sem sinal externo. Para Stechly, Marquez, and Kambhampati (2023), a melhora aparente provém do verificador embutido no protocolo, e Olausson et al. (2024) observam que o auto-reparo em código, a correção do próprio programa a partir do erro observado, só compensa quando o crítico é mais capaz que o gerador. Guiar síntese por contraexemplos antecede os modelos de linguagem. Em CEGIS, sigla de *counterexample-guided inductive synthesis*, um verificador devolve ao sintetizador uma entrada concreta em que o candidato falha (Solar-Lezama et al. 2006). A condição `critic_cegis` transpõe esse protocolo para linguagem natural.

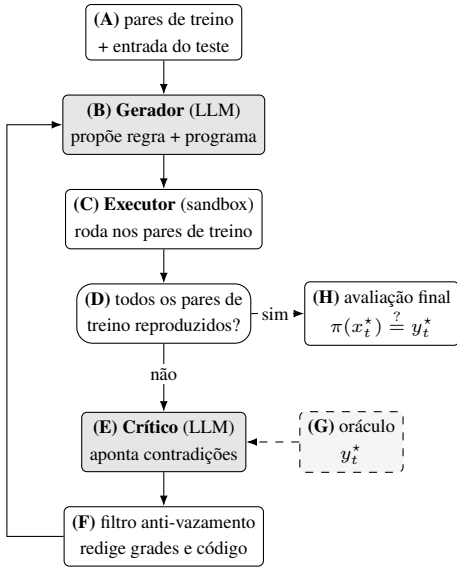


Figura 1: Arquitetura do solucionador. O Gerador (B) nunca vê y_t^* . O Crítico (E) o recebe do oráculo (G), pela linha tracejada, apenas nas condições que o concedem, e sua saída passa pelo filtro (F) antes de voltar ao Gerador. A restrição central é que y_t^* não decide nada. O critério de parada (D) olha apenas para os pares de demonstração, e a avaliação final (H) roda uma única vez, ao término do laço, só para medir.

3 Metodologia

3.1 Formulação e Condições

Uma tarefa t é um par $(\mathcal{D}_t, (x_t^*, y_t^*))$, em que $\mathcal{D}_t$ são os pares de demonstração e (x_t^*, y_t^*) o par de teste. Um solucionador recebe $\mathcal{D}_t$ e x_t^* e produz um programa π em Python. A tarefa é resolvida se e somente se $\pi(x_t^*) = y_t^*$, com igualdade exata da grade. O recurso controlado é o **orçamento de chamadas** B , o número máximo de requisições ao modelo por tarefa, somando as do Gerador e as do Crítico. A hipótese sob teste é que a via iterativa $\mathcal{C}$, com acesso ao oráculo, resolve mais tarefas que a diversificação $\mathcal{S}$ sob o mesmo B ,

$$\Delta = \mathbb{E}_t[\mathbf{1}[\mathcal{C} \text{ resolve } t]] - \mathbb{E}_t[\mathbf{1}[\mathcal{S} \text{ resolve } t]] > 0. \quad (1)$$

Manter B idêntico é o controle experimental central, já que sem ele a condição com mais recursos venceria por dispor de mais recursos.

No laço da Figura 1, $\mathcal{S}$ e $\mathcal{C}$ da Equação 1 correspondem às condições `sampling` e `critic`. `sampling` aloca o orçamento em largura, na estratégia conhecida como *best-of-N*. Cada uma das B chamadas é uma conversa nova com o mesmo *prompt*, sem histórico nem crítica, e vence o candidato que reproduz mais pares de demonstração. `critic` aloca em profundidade, com o Gerador propondo regra e programa, o Crítico apontando contradições em prosa livre e o Gerador revisando com o histórico completo. Cada ciclo de crítica e revisão custa duas chamadas, de modo que com $B = 7$ resultam 3,23 programas contra os 5,43 da

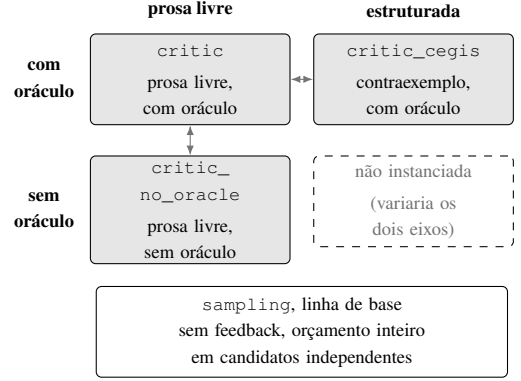


Figura 2: As quatro condições nos dois eixos da decomposição. As setas ligam as comparações que variam um único eixo. Na horizontal muda a forma do feedback, com o acesso ao oráculo constante. Na vertical muda o acesso, com a forma constante.

amostragem. Tomamos *best-of-N* como linha de base porque nada obriga quem dispõe de B chamadas a gastá-las revisando um único programa em série.

A comparação é pragmática e não causal, já que as condições diferem em quatro eixos simultâneos, a existência de feedback, o acesso ao oráculo, o gasto do orçamento em série ou em paralelo e a temperatura (0,8 em `sampling` e 0,2 nas demais, sem o que as N amostras colapsariam em programas quase idênticos). Duas condições adicionais variam um único eixo em relação a `critic` (Figura 2). `critic_no_oracle` recebe o código, a regra e o relatório de execução, mas nunca y_t^* . `critic_cegis` recebe o mesmo que `critic`, inclusive y_t^* , e responde com um contraexemplo e uma classe de correção em vocabulário fechado em vez de prosa livre.

3.2 Salvaguardas e Análise

Três restrições impedem que o acesso ao oráculo contamine a medida. O critério de parada é idêntico nas quatro condições, com o candidato aceito quando reproduz todos os pares de demonstração, e o par de teste roda uma única vez ao final. Se o laço parasse ao acertar o teste, a acurácia mediria o melhor programa que o sistema chegou a produzir, e não o que ele consegue selecionar sozinho. Na saída do Crítico, um filtro redige linhas compostas apenas de inteiros e qualquer código, e contabiliza cada redação, o que torna o vazamento mensurável. O Crítico é reinstituído a cada rodada, sem histórico, e não constrói uma solução ao longo do diálogo. Gerador e Crítico usam o mesmo modelo, `gemin-3.5-flash-lite`, para que uma eventual vantagem não seja atribuível a um segundo modelo mais capaz.

Todas as condições rodam sobre as mesmas 270 tarefas sorteadas de `evaluation` com semente 20260814 e $B = 7$, declaradas antes da execução. Em desenho pareado a quantidade informativa é o número de tarefas que uma condição resolve e a outra não, os pares discordantes. Aplicamos o teste de McNemar exato (McNemar 1947), que descarta as tarefas em que as duas condições concordam

Condição	Res.	Acur.	Treino	Prog.	Tok.
critic	93/270	34,4%	35,6%	3,23	48,1 k
sampling	88/270	32,6%	35,6%	5,43	28,0 k
cegis	83/270	30,7%	34,8%	3,34	48,8 k
no_oracle	80/270	29,6%	33,3%	3,27	45,8 k

Tabela 1: Resultados agregados, ordenados por acurácia no par de teste. A coluna “Treino” dá a fração de tarefas em que o programa final reproduz todos os pares de demonstração, alvo que todas as condições de fato otimizam, já que o oráculo não participa da seleção. “Prog.” é o número médio de programas por tarefa e “Tok.” o consumo médio de tokens.

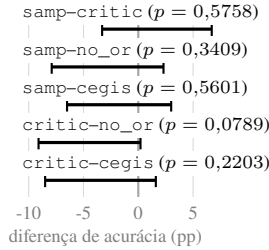


Figura 3: Intervalos de Wilson de 95% das 5 comparações pré-registradas, em pontos percentuais. A linha vertical marca a ausência de efeito e todos os intervalos a cruzam. Os pares discordantes, de cima para baixo, são 51 (23×28), 54 (31×23), 47 (26×21), 47 (30×17) e 54 (32×22). Em 270 tarefas os intervalos ainda admitem efeitos de vários pontos percentuais em qualquer direção.

e avalia se as discordâncias se dividem por acaso. Os intervalos são de Wilson (Wilson 1927) sobre a proporção de discordâncias a favor da condição de crítico, convertidos para pontos percentuais de acurácia, e delimitam a magnitude de efeito compatível com os dados. A família de 5 comparações, cada crítico contra *sampling* e *critic* contra os outros dois, foi pré-registrada, o que exige a correção de Bonferroni para múltiplas comparações (Dunn 1961), com $\alpha = 0,01$ por teste.

4 Resultados

4.1 Comparação em Escala

Nas 270 tarefas (Tabela 1), **nenhuma** das 5 comparações pré-registradas é significativa, nem sob Bonferroni nem a $\alpha = 0,05$ isolado (Figura 3). A comparação mais próxima é *critic* contra *critic_no_oracle*, com $p = 0,0789$ e intervalo de $-9,1$ a $+0,2$ pp. A diferença entre *critic* e *sampling* é de $+1,9$ pp.

O Crítico produz 3,23 programas por tarefa contra 5,43 da amostragem, 40% a menos, atingindo a mesma fração de programas consistentes com os pares de demonstração (35,6% nas duas condições). Não observamos ganho correspondente de acurácia no par de teste. Entre as quatro, *critic_no_oracle* é a única que dispensa o oráculo e portanto a única implantável, e tem a menor acurácia.

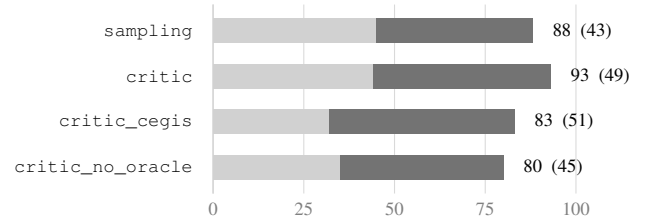


Figura 4: Decomposição das vitórias, em tarefas resolvidas de 270. O segmento claro corresponde às vitórias decididas na primeira geração, comum às quatro condições, e o escuro às obtidas depois dela. Os rótulos dão o total e, entre parênteses, a parcela obtida depois da primeira geração. A ordenação pelo total (93, 88, 83, 80) *inverte* quando se olha apenas essa parcela (51, 49, 45, 43).

4.2 A Origem das Vitórias

A primeira chamada ao modelo é idêntica nas quatro condições, com o mesmo *prompt* e histórico vazio, sem nenhum crítico envolvido, e difere apenas na temperatura. Chamamos de primeira geração o programa que sai dessa chamada. Tarefas resolvidas já ali não distinguem estratégia alguma, porque nenhuma reamostragem ou crítica chegou a ocorrer.

Separando as vitórias da primeira geração das obtidas depois dela (Figura 4), verifica-se que cerca de metade das vitórias de qualquer condição vem do primeiro programa gerado, antes de qualquer crítica ou reamostragem. Restringindo cada par às tarefas que a primeira geração não resolveu em nenhuma das duas condições, todos os 5 p -valores ficam entre 0,2962 e 1,0000.

Contadas apenas as vitórias obtidas depois da primeira geração, as três condições de crítico ficam à frente de *sampling*, ordenação oposta à do total. As diferenças são pequenas e nenhuma é significativa.

O teste pareado se apoia apenas nos pares discordantes, entre 47 e 54 das 270 tarefas conforme o par. Somada à parcela decidida na primeira geração, a evidência capaz de separar as condições vem de uma fração pequena da amostra, bem menor do que n sugere.

Uma medição independente sobre as mesmas tarefas, restrita a *sampling* e *critic*, reproduz o padrão. Nela, 44 das 88 vitórias de *sampling* e 42 das 86 de *critic* vêm da primeira geração, o que deixa o placar empatado em 44×44 depois dela. Nas 206 tarefas que a primeira geração não resolveu, a comparação também não distingue as condições ($p = 0,8877$, intervalo de $-5,6$ a $+4,3$ pp). Sob orçamento igualado, a revisão guiada por oráculo **não** superou a amostragem independente.

Uma correção simples consiste em gravar a primeira geração produzida pela primeira condição a alcançar cada tarefa e reutilizá-la nas demais, com a repetição ainda debitada do orçamento. Nenhuma das medições aqui reportadas adota esse procedimento.

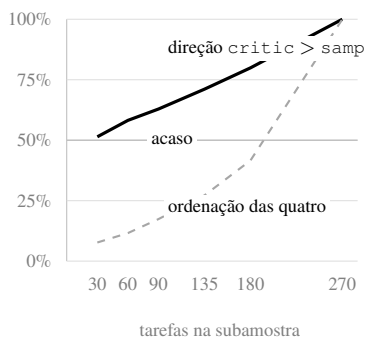


Figura 5: Reprodutibilidade da ordenação sob subamostragem das próprias 270 tarefas, com 20.000 repetições por tamanho, sem reposição e preservando o pareamento. A linha cheia dá a fração de subamostras em que `critic` supera `sampling`, e a tracejada a fração em que a ordenação completa das quatro coincide com a observada em 270. Ambas convergem a 100% em 270 por construção.

4.3 Reprodutibilidade da Ordenação

A ordenação entre as condições é o resultado mais fácil de reter, mesmo sem significância estatística. Para medir quanto dela é reprodutível, subamostramos o conjunto fechado de 270 tarefas sem reposição, preservando o pareamento, em 20.000 repetições por tamanho (Figura 5).

Com 30 tarefas, a direção observada em 270 se reproduz em 51% das subamostras, valor próximo ao acaso, e a banda de 5 a 95% da diferença de acurácia vai de $-10,0$ a $+13,3$ pp. Com 60 tarefas o valor sobe para 58%. A ordenação completa das quatro condições se reproduz em 8% e 12% das subamostras desses dois tamanhos. Como a subamostragem capta apenas variância de amostra, esses valores subestimam a instabilidade de execuções independentes.

Uma análise interina feita durante a coleta ilustra o mesmo risco. Sobre as primeiras 91 tarefas, projetar linearmente as proporções da comparação mais próxima dava $p = 0,0027$, valor que teria atravessado a correção de Bonferroni, contra $p = 0,0789$ no conjunto fechado (Button et al. 2013).

5 Discussão e Limitações

O oráculo dá à via iterativa uma vantagem que nenhum sistema implantável teria, e ainda assim as condições não se separam. Não encontramos evidência de que o acesso à informação privilegiada seja o que falta ao canal de feedback, o que enfraquece essa explicação para os ganhos relatados na literatura. Os intervalos ainda admitem efeitos de vários pontos percentuais em qualquer direção, de modo que os resultados não demonstram equivalência entre as condições.

O confundimento da primeira geração não é específico deste arranjo. Qualquer comparação entre esquemas de inferência que partam do mesmo *prompt* inicial o herda, e ele não aparece na acurácia agregada.

Contabilizamos o orçamento em chamadas. As condições de crítico consomem de 63% a 74% mais tokens por tarefa, de modo que sob um orçamento medido em tokens elas fariam menos iterações e a comparação lhes seria menos favorável.

Limitações. Metade do resultado vem de uma chamada comum às condições, e a correção descrita acima não foi aplicada. `critic` e `critic_cegis` consultam o oráculo durante o laço, de modo que medem o valor da validação por oposição em condições favoráveis e não o de um método implantável. A linha de base foi coletada em data anterior à das condições de crítico, e o modelo é hospedado, o que introduz nas comparações com `sampling` um confundimento temporal ausente nas comparações entre críticos. Cada tarefa foi executada uma vez, e o pareamento mitiga o ruído do modelo sem eliminá-lo. As conclusões referem-se ao regime de capacidade de `gemin-3.5-flash-lite`.

6 Conclusão

Em 270 tarefas do ARC-AGI-1 sob orçamento fixo de chamadas, quatro condições que variam isoladamente o acesso ao oráculo e a forma do feedback não se separam, e **nenhuma** das 5 comparações pré-registradas é significativa. O Crítico atinge a mesma fração de programas consistentes com os pares de demonstração usando 40% menos programas, sem ganho correspondente de acurácia no teste. Conceder o gabarito ao Crítico não bastou para que a via iterativa superasse a amostragem, o que enfraquece a explicação pelo canal de feedback sem descartá-la.

Um segundo resultado é que cerca de metade das vitórias de qualquer condição vem do primeiro programa gerado, antes de qualquer crítica ou reamostragem. Descontada essa parcela, a ordenação entre as condições se inverte. A ordenação observada nas 270 tarefas se reproduz em apenas 12% das subamostras de 60. Antes de aumentar n , trabalhos futuros devem eliminar esse confundimento por construção e contabilizar o orçamento em tokens.

Referências

- Brown, B.; Juravsky, J.; Ehrlich, R.; Clark, R.; Le, Q. V.; Ré, C.; and Mirhoseini, A. 2024. Large Language Monkeys: Scaling Inference Compute with Repeated Sampling. arXiv:2407.21787.
- Button, K. S.; Ioannidis, J. P. A.; Mokrysz, C.; Nosek, B. A.; Flint, J.; Robinson, E. S. J.; and Munafò, M. R. 2013. Power Failure: Why Small Sample Size Undermines the Reliability of Neuroscience. *Nature Reviews Neuroscience*, 14(5): 365–376.
- Chen, M.; Tworek, J.; Jun, H.; Yuan, Q.; Pinto, H. P. d. O.; Kaplan, J.; Edwards, H.; Burda, Y.; Joseph, N.; Brockman, G.; et al. 2021. Evaluating Large Language Models Trained on Code. arXiv:2107.03374.
- Chen, X.; Lin, M.; Schärli, N.; and Zhou, D. 2024. Teaching Large Language Models to Self-Debug. In *The Twelfth International Conference on Learning Representations (ICLR)*.
- Chollet, F. 2019. On the Measure of Intelligence. arXiv:1911.01547.
- Dunn, O. J. 1961. Multiple Comparisons Among Means. *Journal of the American Statistical Association*, 56(293): 52–64.
- Huang, J.; Chen, X.; Mishra, S.; Zheng, H. S.; Yu, A. W.; Song, X.; and Zhou, D. 2024. Large Language Models Can-

not Self-Correct Reasoning Yet. In *The Twelfth International Conference on Learning Representations (ICLR)*.

Li, Y.; Choi, D.; Chung, J.; Kushman, N.; Schrittwieser, J.; Leblond, R.; Eccles, T.; Keeling, J.; Gimeno, F.; Dal Lago, A.; et al. 2022. Competition-Level Code Generation with AlphaCode. *Science*, 378(6624): 1092–1097.

Madaan, A.; Tandon, N.; Gupta, P.; Hallinan, S.; Gao, L.; Wiegrefe, S.; Alon, U.; Dziri, N.; Prabhume, S.; Yang, Y.; et al. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 46534–46594.

McNemar, Q. 1947. Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages. *Psychometrika*, 12(2): 153–157.

Olausson, T. X.; Inala, J. P.; Tan, C.; Gao, J.; and Solar-Lezama, A. 2024. Is Self-Repair a Silver Bullet for Code Generation? In *The Twelfth International Conference on Learning Representations (ICLR)*.

Shinn, N.; Cassano, F.; Berman, E.; Gopinath, A.; Narasimhan, K.; and Yao, S. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 8634–8652.

Snell, C.; Lee, J.; Xu, K.; and Kumar, A. 2024. Scaling LLM Test-Time Compute Optimally Can Be More Effective Than Scaling Model Parameters. arXiv:2408.03314.

Solar-Lezama, A.; Tancau, L.; Bodik, R.; Seshia, S.; and Saraswat, V. 2006. Combinatorial Sketching for Finite Programs. In *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 404–415.

Stechly, K.; Marquez, M.; and Kambhampati, S. 2023. GPT-4 Doesn’t Know It’s Wrong: An Analysis of Iterative Prompting for Reasoning Problems. arXiv:2310.12397.

Wilson, E. B. 1927. Probable Inference, the Law of Succession, and Statistical Inference. *Journal of the American Statistical Association*, 22(158): 209–212.